

Dead Signal

Technical Design

Jharek

Version 1.1 · 2026-07-06

Table of Contents

Version

Status

Last updated

Owner

Companion docs

Purpose. The engineering reference for how Dead Signal is built: architecture, the phase state machine, the data model, the runtime systems, the save schema, audio, and build/deploy. The code is the ultimate source of truth; this doc explains its shape and the *why* behind the load-bearing decisions. For *what it is* see the [PRD](#); for *how it plays* see the [GDD](#).

Table of contents

1. [Architecture overview](#)
 2. [The phase state machine](#)
 3. [Data model — content as data](#)
 4. [Runtime systems](#)
 5. [Tuning constants](#)
 6. [Audio engine](#)
 7. [Persistence & save schema](#)
 8. [Presentation layer](#)
 9. [Build, PWA & deployment](#)
 10. [Performance](#)
 11. [Testing & QA](#)
 12. [Extensibility — how to add content](#)
 13. [Known tech debt](#)
 14. [Appendix](#)
-

1. Architecture overview

Dead Signal is a **single-file React game component** plus a small procedural audio module. There is no game engine, no ECS, no external state library — the whole game is React state, module-scope authored data, and a set of hand-rolled timer queues.

```
src/
├── main.jsx      App mount (StrictMode) + window.storage → localStorage shim
├── DeadSignal.jsx THE GAME — all state, systems, content, rendering (~5,165
lines)
├── audio.js      Tone.js procedural audio singleton (~174 lines)
└── demo/         Marketing/browser landing shell + embedded game wrapper
```

Load-bearing decisions

- **Single component, by choice.** Everything lives in DeadSignal.jsx. This keeps the (large) game in one navigable file and avoids prop-drilling a deeply interdependent state graph. The trade-off is file size; mitigated by clear module-scope data blocks at the top and a consistent section ordering.
- **State mirrored into refs for timer-safe async.** The game is driven by setTimeout-based message scheduling. React state is asynchronous and closures capture stale values, so **every piece of state that a timer callback must read is mirrored into a useRef** (gamePhaseRef, resourcesRef, noiseRef, currentPathRef, discoveredTruthsRef, ...). The pattern: setX(v) for render + xRef.current = v (via an effect) for timers. This is the single most important thing to understand before editing runtime logic.
- **Authored content is data, not code.** Beats, pools, encounters, regions, weapons, route profiles, and the Case File are declared as **module-scope constant objects/arrays** at the top of the file. Runtime systems interpret this data. Adding content is (mostly) editing data, not logic (§12).
- **Determinism.** No network, no AI, no unseeded randomness the player can't reason about. Random picks (pickRandom) are for variety (which atmosphere line), deduped per run via seen*Ref sets; they never change outcomes the player is asked to reason about (encounter odds are surfaced).
- **Additive audio.** audio.js is a module-scope singleton; every method is a **safe no-op** until unlocked and while muted, so game code can call audio freely without guards.

2. The phase state machine

gamePhase (mirrored to gamePhaseRef) is the primary driver. Named phases and their role:

Phase	Role
phase1	Apartment: wake, contact, prep, route

p2_ai	selection. Battery drain = 0 here. Phase-2 path leg (the chosen route). Advancing beats drain 1%.
p2_memory_frag	Memory-fragment beat (drain 0).
p2_discovery	Route-clue discovery beat (drain 0).
p2_ai_cross	The crossing leg (after the route leg).
p2_scripted	Scripted-intro sub-phase of the crossing (set on entry, then hands to p2_ai); has its own offline-coherence guard.
encounter	Hazard interrupt; resolves and returns to the caller leg via a return-phase.
shelter	Day transition / setup / resource beat.
haven_approach	Approach beats; first Signal-distortion; the crack.
haven_ai	Haven exploration; guaranteed battery top- up on entry.
haven_final	The empty-Haven finale of the prologue; the 143 record; the call.
phase3	The investigation: Haven hub + region nodes (mercy/comms/cityhall/annex).
phase3_finale	Final call + Accept/Refuse + endings.

Transitions of note. The prologue **auto-flows** haven_final → phase3 (there is no standalone “to be continued” screen — it was culled; see STORY.md changelog #8). Encounters are re-entrant: they store a return-phase and restore the exploration leg that spawned them. A sync gotcha is documented in PLAN.md M3 — sites that setGamePhase(...) then call the same tick must pass a phaseOverride because the ref hasn’t updated yet.

3. Data model — content as data

All of the following are **module-scope constants** at the top of DeadSignal.jsx. This is the authored surface of the game.

Narrative & beats

- INTRO_LINES — the opening apartment lines.
- PATH_BEATS — per-route scripted beats.
- MEMORY_FRAGMENT_POOLS — 3 pools × 3 fragments = **9**
(ALL_FRAGMENT_NAMES, FRAGMENT_BY_NAME index the flat set so board
drop-downs can replay a flashback).
- DISCOVERY_BEATS — the route clues.

- EXPLORE_BEATS / STATE_LINES — atmosphere pools + resource-state lines (battery_critical, battery_low, injured_bad, low_food, low_water), gated to avoid spam.
- HAVEN_APPROACH_BEATS, HAVEN_DESTINATIONS, HAVEN_RECORDS_BEAT, HAVEN_FINAL_SEQUENCE — the Haven set-pieces.
- FINALE_CONVERGENCE, FINALE_ACCEPT / FINALE_REFUSE, ACCEPT_ENDING_LINES / REFUSE_ENDING_LINES, and the choice constants (FINALE_CHOICE, ACCEPT_CHOICE, REFUSE_CHOICE).
- BRANCH_PATHS, NARRATOR_ATMOSPHERE, ELLIE_DEFLECT, ELLIE_NOISE, OFFLINE_LINES, DEATH_LINES.

Investigation & regions

- REGIONS — the 5 Phase-3 regions' **metadata**, each { key, name, truth, truthId, reveal(clues, reached, path), blurb }. truthId is what's stored in discoveredTruths; truth is the prose label.
- PHASE3_REGIONS — the **node maps** for those regions (per-node onEnter/revisit/exits/ caseFile). Region *nodes* live here; region *metadata/reveal* lives in REGIONS above. (Both are real, shipped constants — don't conflate them.)
- TRUTH_UNLOCKS = { you: "cityhall" } — truth-gated region unlocks.
- BOARD_CLUES, BOARD_PEOPLE, BOARD_FACTS, BOARD_CONTRADICTIONS, BOARD_QUESTIONS (BASE_QUESTION_TEXT derived) — the Case File data. Facts/contradictions share a **reveal signature** (clues:Set, reached:boolean, raised:string[]) → boolean, so a fact appears when its predicate is satisfied by collected clues + world flags.

Systems data

- ENCOUNTERS, CORNERED_ENCOUNTER, ENCOUNTER_REACTIONS — hazard definitions + reaction lines.
 - WEAPONS (knife 2 → machete 6), WEAPON_PICKUPS — the weapon ladder.
 - ROUTE_PROFILE — per-route powerBias / noiseCombatPenalty / noiseDecayPerLeg.
 - SEARCH_LOOT, POWER_SOURCES (a Set), BRIDGES, EXPLORE_LABELS / EXPLORE_DONE / MOVE_ON_LABEL, CALM_BEAT.
 - buildLeadQueue(section) — returns the ordered lead list for a leg (path / crossing / haven).
-

4. Runtime systems

4.1 Message scheduling (the heartbeat)

scheduleMessages owns timed message reveals; its timers live in dialogueRef, kept **separate** from pendingRef (the general pending-timer set) so dialogue can be paused/flushed independently (PLAN.md/comments C3). idRef is a monotonic id source (avoids Date.now() key collisions, H1). Human-paced texting delay is intentional (instant

replies feel cheap). **Pause** freezes the queue mid-beat and resumes it, which is both a UX and an accessibility feature.

4.2 Battery & resource model

beatBatteryCost(phase) centralizes drain: phase1 → 0, p2_memory_frag/p2_discovery → 0, continues (·) → 0, else → 1%/beat. The **charger** is a rechargeable reservoir: CHARGER_FIND(20) dumps into the phone in P1; power-source searches add CHARGER_RECHARGE(25) to the reservoir; the **free** “Use charger” action transfers CHARGER_TRANSFER(25) reservoir→phone (gated: reservoir non-empty, phone < 90%, not mid-encounter). POWER_SOURCES is the set of search nodes that recharge. The Haven cache (HAVEN_BATTERY_CACHE=45, HAVEN_SUPPLY_FLOOR=5) is the end-of-gauntlet relief, placed diegetically. Offline (battery 0) and starvation (food/water 0) are death states with their own line pools (OFFLINE_LINES, DEATH_LINES).

4.3 Encounter resolver

A shared resolver computes pSneak / pRun / pFight from **noise + weapon**, and the **displayed risk tier** ([LOW]/[MED]/[HIGH]) is derived from those *same* functions — one source of truth for the roll and the tag (PLAN.md-era changelog #9). FORCE is a no-roll [COSTLY] guaranteed outcome. RISK_TOKEN_RE strips at most one authored [risk] token per choice; a tag is injected only when a quiet option’s odds degrade to MED/HIGH. Route identity feeds in via ROUTE_PROFILE.noiseCombatPenalty and noiseDecayPerLeg.

4.4 The lead-queue walker

buildLeadQueue(section) produces the ordered leads; the leg handler walks them with a cursor (the exchange count). Every exploration screen offers “explore” (next lead) and — once the required discovery (route clue) is found — “move on”. atmo may fire a transition drain; encounter carries a plan (power/search/hazard); calm is the one-per-run breather (calmFired, save-persisted; never fires in Phase 3). This replaced scattered newCnt==4/fragTarget counters with one declarative, serializable structure (PLAN.md M8).

4.5 Case File reveal engine

Board facts, contradictions, and questions are evaluated against (clues, reached, raised). raised is a set of world-flag keys pushed by beat/node hooks (haven143, p3_mercy_truth, p3_comms_loop, ...). A fact shows when its predicate passes; a **contradiction** pairs two known facts into the question they force; **questions** can evolve into a sharper form (kim → kim143; haven → haven143) and announce via NEW/UPDATED QUESTION cards (evolution keys are excluded from BASE_QUESTION_TEXT so they don’t double-announce).

4.6 Truth tracking & Phase-3 gating

discoveredTruths (array of truthId, mirrored to discoveredTruthsRef) is the spine of Phase-3 progression:

- **Truth-gate:** TRUTH_UNLOCKS[truthId] unlocks a region when that truth lands (e.g. you → City Hall).
- **Count-gate:** the Research Annex unlocks at discoveredTruthsRef.current.length ≥ 2 .
- **Finale-gate:** the finale appears at Haven's gate_yard when length ≥ 4 .

phase3Unlocked / visitedPhase3Nodes track region availability and node visits.

Expansion v2 (build: PLANNED). Truth-by-assembly (S1) will gate each truth's raise behind its 2–3 supporting raised flags via the existing reveal predicate (§4.5) — no new gating engine. See §12.1 for the full planned mapping.

5. Tuning constants

The balance knobs, all module-scope in DeadSignal.jsx. **This table is the canonical reference** cited by the GDD and PRD.

Constant	Value	Meaning	Notes
beatBatteryCost(p hase)	0 or 1	Battery drain per beat	1%/beat on advancing P2/P3 beats; 0 in phase1, memory/discovery, continues
CHARGER_FIND	20	P1 charger find → phone	Starts Phase 2 at ~29%; tight on purpose
CHARGER_RECHARGE	25	Reservoir gained per power-source search	
CHARGER_TRANSFER	25	Reservoir → phone per “Use charger” tap	Free action
START_SUPPLY	4	Starting food & water	Neglect → 0 → death; searching keeps margin
HAVEN_BATTERY_CACHE	45	Battery from Haven ops-building cache	Replaced an invisible floor
HAVEN_SUPPLY_FLOOR	5	Haven pantry tops food/water to $\geq$ this	
WEAPONS[*].damage	2–6	knife 2 · bat 3 · crowbar 4 · axe 5 · machete 6	Feeds pFight

ROUTE_PROFILE. powerBias	+0.20 / -0.05 / -0.25	hospital / metro / route9 power frequency	
ROUTE_PROFILE. noiseCombatPenal ty	0.18 / 0.10 / 0.05	hospital / metro / route9	Tight halls carry sound
ROUTE_PROFILE. noiseDecayPerLeg	0 / 1 / 2	hospital / metro / route9	Outdoors disperses sound
SLOT_COUNT	3	Save slots	
MAX_VISIBLE_CH OICES / HARD_CHOICE_C AP	4 / 5	Choice UI limits	
NOTIF_DELAY	11400 ms	Notification timing	
DAY_GATE_MS / EARLY_WAKE_MI N_MS	17 min / 2 min	Vestigial day-gate timing	Real-time gates were dropped (STORY.md #8)

Tuning philosophy. Battery is the load-bearing knob. To make ignore-power outright lethal before Haven, lower the post-charger budget (CHARGER_FIND); the other knobs shape comfort, not the death line. Phase 3 currently inherits the P2 economy (open lever — GDD \$15).

6. Audio engine

src/audio.js is a **module-scope Tone.js singleton**. Design constraints (from the file header):

- **No looping ambient bed.** Only short UI/feedback one-shots plus one quiet resolve chord on completion. Silence is the aesthetic (GDD Pillar 5).
- **Lazy + gesture-gated.** Tone.js (~380 kB) is dynamically import()-ed in unlock() on the first pointer/key gesture, keeping it out of the initial bundle. Nodes are built **once** and reused.
- **Additive & safe.** Every method no-ops until unlocked and while muted, so game code calls audio without guards.

Signal chain: one-shot voices → master gain → Limiter(-2) → destination; a Reverb tail feeds the completion chord. MASTER_DB = -4 headroom; user volume (0–1) and muted layer on top via applyMasterGain.

Voices / methods:

Method	Sound	Where
tapResponse() /	UI taps (triangle/sine)	Buttons, menus

tapMenu()		
blip()	Message arrival (quiet sine)	Incoming texts
gain() / loss()	Resource stings (rising / falling)	Loot / damage
terminal("complete")	Consonant resolve chord (C-E-G-B)	Run completion
signal()	Signal distortion — pink-noise crackle + detuned two-note chirp	Story-gated only

iOS handling. unlock() awaits Tone.start(), confirms the context actually resumed (retries resume()), and bails *without* marking unlocked if it can't — so the next gesture retries. It deliberately does **not** set navigator.audioSession.type = "playback" (that leaks audio to the background and wedges the context). resume() is called on return-to-foreground.

The **Audio Bible** (../audio/) will cover palette intent, the Signal-cue placement rules, and mix philosophy in full.

7. Persistence & save schema

Storage API. The game persists via window.storage.{get,set,delete} — an async API from the Claude artifact sandbox (get() → { value } or null). Standalone builds shim it onto localStorage in main.jsx, mirroring the same shape.

Slots & shape. SLOT_COUNT = 3, keyed by slotKey(i). Each slot stores buildSlotData(profile, runSnapshot):

- **Profile** (per-slot, persistent progression): playthroughs, accumulated fragments, clues, ending, and progress toward 100%.
- **Run snapshot** (optional, resumable mid-run): current phase/path, resources, noise, weapon, discoveredTruths, recovered memories, Case File state, and a meta block ({ day, location, hp, battery, savedAt }) for the load menu.

Versioning & migration. Saves are versioned (raw.v). A legacy bare snapshot (v === 1) is migrated to a run + empty profile on load. One-time migrations move the old global ds_memories and ds_save into the per-slot model. **Forward-compatible:** missing fields default, so older saves load into newer builds (verified in PLAN.md M4).

Non-resumable mid-beat states. A snapshot with no choices up and no parked gate doesn't resume mid-beat — the game restores to a safe decision point. Settings (ds_muted, ds_volume) persist separately.

8. Presentation layer

- **Chat rendering** — a memoized message list (memo), auto-scroll with save/restore of scroll position (chatScrollRef, restoreChatScrollRef, suppressNextAutoScrollRef) so returning from the Case File or pause doesn't jump.
 - **HUD** — signal + battery corner-anchored; state-derived locationLabel(); resource readouts; pulseBattery() feedback on transfers.
 - **Choices** — intent-coded styling; capped at MAX_VISIBLE_CHOICES/HARD_CHOICE_CAP; HUD-register tags (risk tiers, [$\pm$ N Resource]).
 - **Voice separation** — Ellie = lowercase message bubbles; narrator = italic centered — protecting the phone frame (GDD Pillar 2).
 - **Screens** — title/menus, three-slot save/load, Options (volume+mute), spoiler-safe Story page, in-game pause (freezes dialogue), the Case File board+journal.
 - **Document lock** — index.html fixes html,body,#root and position:fixed the body to kill iOS rubber-band scroll; the app manages its own scroll panes.
 - **Debug** — an on-screen ?debug overlay surfaces the audio status() snapshot.
-

9. Build, PWA & deployment

- **Build:** Vite 5 (npm run dev / build / preview). Production build verified clean (PLAN.md M4: 994 modules, 0 errors).
 - **PWA:** manifest.webmanifest, generated icons (npm run icons → scripts/gen-icons.mjs using @resvg/resvg-js from public/icon.svg), iOS home-screen meta in index.html.
 - **Entry points:** index.html (game) and demo.html (marketing/browser shell; #play route runs the embedded game).
 - **Deploy:** GitHub Pages via .github/workflows/deploy.yml.
 - **Dead infra to note:** the old vite.config.js API proxy and .env* VITE_ANTHROPIC_KEY are now unused (AI removal, PLAN.md M4).
-

10. Performance

- **Bundle:** Tone.js is lazy-loaded off the critical path; the initial bundle is React + the game.
 - **Timers:** message scheduling is setTimeout-based; timers are tracked in ref sets and cleared on pause/unmount/reset to avoid leaks and double-fires.
 - **Rendering:** the message list is memoized; scroll is managed manually to stay smooth on mobile.
 - **Targets:** fast boot on mid-range mobile, smooth scroll on small screens (PRD NFR-2).
-

11. Testing & QA

- **Determinism makes QA tractable** — outcomes are reproducible. The core regression is a **full start→ending playthrough per route** with **zero console errors** and **zero network calls** (the standing bar from PLAN.md).
 - **In-page scripted driver.** Playthroughs were verified via a scripted in-page driver walking beats end to end (PLAN.md M3) — the basis for an automated smoke test.
 - **Save round-trip.** Save → reload → resume must restore a safe state; legacy saves must migrate.
 - **Platform matrix.** iOS Safari + Android Chrome, installed PWA + in-browser; audio must unlock or safely no-op.
 - **Content QA.** A consistency pass against STORY.md (canon), and a spoiler audit of public surfaces, are release gates (PRD §8).
-

12. Extensibility — how to add content

Because content is data (§3), most additions are data edits:

- **Add an atmosphere beat** — append to the relevant EXPLORE_BEATS pool. Deduped per run automatically.
- **Add a memory fragment** — add to a MEMORY_FRAGMENT_POOLS route pool; the flat indexes (ALL_FRAGMENT_NAMES, FRAGMENT_BY_NAME) and the board pick it up. Keep the count coherent (9).
- **Add an encounter** — add to ENCOUNTERS; reference it from a lead's plan. The shared resolver handles odds/tags.
- **Add a Case File fact/question** — add to BOARD_FACTS/BOARD_QUESTIONS with a reveal predicate over (clues, reached, raised); raise its flag from the beat/node that earns it.
- **Add a Phase-3 region** — add to REGIONS (key, truthId, reveal, gating); wire its nodes and the truth payoff; add any TRUTH_UNLOCKS entry or count-gate.
- **Tune balance** — edit the §5 constants; re-run the ignore-power measurement.

Rules when editing: respect STORY.md canon and spoiler discipline; keep new timer-read state mirrored into a ref (§1); never leave the build broken between changes.

12.1 Expansion v2 — planned implementation shape (build: PLANNED)

Expansion v2 (STORY.md “Changelog — Expansion v2”) is **authored content plus one Case File extension**, not new systems — it maps onto the existing data model (§3) as below. **None of this is in the code yet**; this is the target shape for the later implementation pass, recorded so the docs stay accurate about what is and isn't built.

- **Echoes — data.** A new module-scope ECHOES table (fragments keyed by resident, each carrying the region/node that surfaces it and its 2–4 authored lines). Recovery state joins the profile + run snapshot as an echoesRecovered set (mirrored to a ref like every timer-read set, §1) and accumulates toward 100% like fragments/clues

(§7). A new **ECHOES board category** renders alongside MEMORIES/CLUES. Surfacing reuses the existing node caseFile.raise hook — **no new runtime system**.

- **New nodes → data.** The ~30 new Phase-3 rooms are appended to the node map (PHASE3_REGIONS — the per-node data, distinct from the REGIONS metadata list; §3) exactly like the shipped nodes (onEnter/revisit/exits/caseFile), roughly doubling node count (~35 → ~65). **No new phase** in the state machine (§2); phase3 and the region ids are unchanged.
- **Truth-by-assembly (S1) → reveal predicate.** A region's truth is raised on its truth node today; S1 gates that raise behind its 2–3 supporting raised flags. The Case File reveal engine (§4.5) already evaluates predicates over (clues, reached, raised), so this is a predicate change, not a new engine. The evolving question why143 → who_left_out uses the existing evolve mechanism (§4.5).
- **The unchosen (U1) → content.** Authored into City Hall's nodes + the Marcus/Rosa Echoes + the evolving question above. No new mechanics.
- **Prologue second act (P1–P3) → lead queue + beats.** Route midpoints extend buildLeadQueue("path") with extra atmo/discovery leads; the shelter scene and additional Ellie cracks are new beats in the existing prologue phases. Battery economy (§4.2) and spoiler discipline unchanged; no Echoes in the prologue.
- **Held OPEN: U2** (a new region → one more PHASE3_REGIONS entry + gating) and **X1** (ending variation keyed on echoesRecovered) are documented but out of this pass.

Cross-cutting: the Echo-recovery audio cue is a planned signal() sibling one-shot (§6 / Audio Bible); this roughly **doubles Phase-3 string volume**, so localization externalization (§13) should precede it.

13. Known tech debt

From the PLAN.md follow-ups and the code comments:

- **Vestigial state:** fragTarget + sectionPlanRef are no longer read (superseded by the lead queue) but still in the snapshot/reset — safe to remove in a cleanup pass.
 - **Dead infra:** vite.config.js API proxy and .env* VITE_ANTHROPIC_KEY (AI removal residue).
 - **applyChoiceLoot** still returns a now-unused truth field.
 - **Vestigial day-gate timing** (DAY_GATE_MS, EARLY_WAKE_MIN_MS) — real-time gates were dropped.
 - **Localization:** strings are hardcoded inline — externalization is the prerequisite for F-19 (PRD) and is the largest single refactor on the horizon.
 - **File size:** DeadSignal.jsx is large by design but approaching the point where extracting the pure data blocks into a sibling module (still single-import) would aid navigation without changing architecture.
-

14. Appendix

14.1 File map

Path

src/DeadSignal.jsx
src/audio.js
src/main.jsx
src/demo/
index.html / demo.html
public/
scripts/gen-icons.mjs
.github/workflows/deploy.yml

14.2 The ref-mirroring pattern (reference)

// State for render:

```
const [gamePhase, setGamePhase] = useState("phase1");
```

// Ref for timers (async-safe read):

```
const gamePhaseRef = useRef(gamePhase);
```

```
useEffect(() => { gamePhaseRef.current = gamePhase; }, [gamePhase]);
```

// In a timer callback, ALWAYS read gamePhaseRef.current, never gamePhase.

14.3 Related documents

- **Design:** GDD · **Product:** PRD
- **Canon:** STORY.md · **Historical:** PLAN.md
- **Audio Bible:** ../audio/ (planned)

14.4 Change log

Version	Date	Notes
1.0	2026-07-06	First technical design doc. Grounded in DeadSignal.jsx (~5,165 lines) + audio.js.
1.1	2026-07-06	Added §12.1 — Expansion v2 planned implementation shape (Echoes/echoesRecovered , ~30 new PHASE3_REGIONS nodes, truth-by-assembly as a reveal-predicate change, prologue lead-queue extension) + §4.6 pointer. All build: PLANNED; no

code changed.

End of document.